
Detection and Prevention of SQL Injection Attacks

William G.J. Halfond and Alessandro Orso

Georgia Institute of Technology
{whalfond, orso}@cc.gatech.edu

Summary. We depend on database-driven web applications for an ever increasing amount of activities, such as banking and shopping. When performing such activities, we entrust our personal information to these web applications and their underlying databases. The confidentiality and integrity of this information is far from guaranteed; web applications are often vulnerable to attacks, which can give an attacker complete access to the application's underlying database. SQL injection is a type of code-injection attack in which an attacker uses specially crafted inputs to trick the database into executing attacker-specified database commands. In this chapter, we provide an overview of the various types of SQL injection attacks and present AMNESIA, a technique for automatically detecting and preventing SQL injection attacks. AMNESIA uses static analysis to build a model of the legitimate queries an application can generate and then, at runtime, checks that all queries generated by the application comply with this model. We also present an extensive empirical evaluation of AMNESIA. The results of our evaluation indicate that AMNESIA is, at least for the cases considered, highly effective and efficient in detecting and preventing SQL injection attacks.

5.1 Introduction

SQL Injection Attacks (SQLIAs) have emerged as one of the most serious threats to the security of database-driven applications. In fact, the Open Web Application Security Project (OWASP), an international organization of web developers, has placed SQLIAs among the top ten vulnerabilities that a web application can have [7]. Similarly, software companies such as Microsoft [3] and SPI Dynamics have cited SQLIAs as one of the most critical vulnerabilities that software developers must address. SQL injection vulnerabilities can be particularly harmful because they allow an attacker to access the database that underlies an application. Using SQLIAs, an attacker may be able to read, modify, or even delete database information. In many cases, this information is confidential or sensitive and its loss can lead to problems such as identity theft and fraud. The list of high-profile victims of SQLIAs includes Travelocity, FTD.com, Creditcards.com, Tower Records, Guess Inc., and the Recording Industry Association of America (RIAA).

The errors that lead to SQLIAs are well understood. As with most code-injection attacks, SQLIAs are caused by insufficient validation of user input. The vulnerability occurs when input from the user is used to directly build a query to the database. If the input is not properly encoded and validated by the application, the attacker can inject malicious input that is treated as additional commands by the database. Depending on the severity of the vulnerability, the attacker can issue a wide range of SQL commands to the database. Many interactive database-driven applications, such as web applications that use user input to query their underlying databases, can be vulnerable to SQLIA. In fact, informal surveys of database-driven web applications have shown that almost 97% are potentially vulnerable to SQLIA.

Like most security vulnerabilities, SQLIAs can be prevented by using defensive coding. In practice however, this solution is very difficult to implement and enforce. As developers put new checks in place, attackers continue to innovate and find new ways to circumvent these checks. Since the state of the art in defensive coding is a moving target, it is difficult to keep developers up to date on the latest and best defensive coding practices. Furthermore, retroactively fixing vulnerable legacy applications using defensive coding practices is complicated, labor-intensive, and error-prone. These problems motivate the need for an automated and generalized solution to the SQL injection problem.

In this chapter we present AMNESIA (Analysis and Monitoring for NEutralizing SQL Injection Attacks), a fully automated technique and tool for the detection and prevention of SQLIAs.¹ AMNESIA was developed based on two key insights: (1) the information needed to predict the possible structure of all legitimate queries generated by a web application is contained within the application's code, and (2) an SQLIA, by injecting additional SQL statements into a query, would violate that structure. Based on these two insights we developed a technique against SQL injection that combines static analysis and runtime monitoring. In the static analysis phase, AMNESIA extracts from the web-application code a model that expresses all of the legitimate queries the application can generate. In the runtime monitoring phase, AMNESIA checks that all of the queries generated by the application comply with the model. Queries that violate the model are stopped and reported.

We also present an extensive empirical evaluation of AMNESIA. We evaluated AMNESIA on seven web applications, including commercial ones, and on thousands of both legitimate and illegitimate accesses to such applications. We modeled the illegitimate accesses after real attacks that are in use by hackers and penetration testing teams. In the evaluation, AMNESIA did not generate any false positives or negatives and had a very low runtime overhead. These results indicate AMNESIA is an effective and viable technique for detecting and preventing SQLIAs.

The rest of the chapter is organized as follows. Section 5.2 discusses SQLIAs and their various types. Section 5.3 illustrates our technique against SQLIAs. Section 5.4 presents an empirical evaluation of our technique. Section 5.5 compares our approach to related work. Section 5.6 concludes and discusses future directions for the work.

¹ An early version of this work was presented in [9].

5.2 SQL Injection Attacks Explained

The presence of an SQL injection vulnerability allows an attacker to issue commands directly to a web application's underlying database and to subvert the intended functionality of the application. Once an attacker has identified an SQLIA vulnerability, the vulnerable application becomes a conduit for the attacker to execute commands on the database and possibly the host system itself.

SQLIAs are a class of code injection attacks that take advantage of a lack of validation of user input. The vulnerabilities occur when developers combine hard-coded strings with user-input to create dynamic queries. If the user input is not properly validated, it is possible for attackers to shape their input in such a way that, when it is included in the final query string, parts of the input are evaluated as SQL keywords or operators by the database.

5.2.1 Example of an SQLIA

To illustrate how an SQLIA can occur, we introduce an example web application that is vulnerable to a type of SQLIA that we call a tautology-based attack. The architecture of this web application is shown in Figure 5.1. In this example, the user interacts with a web form that takes a login name and pin as input and submits them to the web server. The web server passes the user supplied credentials to a servlet (`show.jsp`, in the example), which resides on the application server. The servlet queries the database to check whether the credentials are valid and, based on the result of the query, generates a response for the user in the form of a web page. The servlet, whose partial implementation is shown in Figure 5.2, uses the user-supplied credentials to dynamically build a database query. Method `getUserInfo` is called with the login and pin provided by the user. If both `login` and `pin` are empty, the method submits the following query to the database:

```
SELECT info FROM users WHERE login='guest'
```

Conversely, if `login` and `pin` are specified by the user, the method embeds the submitted credentials in the query. Therefore, if a user submits `login` and `pin` as “doe” and “123,” the servlet dynamically builds the query:

```
SELECT info FROM users WHERE login='doe' AND pin=123
```

A web site that uses this servlet would be vulnerable to SQLIAs. For example, if a user enters “' OR 1=1 --” and “”, instead of “doe” and “123”, the resulting query is:

```
SELECT info FROM users WHERE login='' OR 1=1 --' AND pin=
```

The database interprets everything after the `WHERE` token as a conditional statement and the inclusion of the “`OR 1=1`” clause turns this conditional into a tautology. (The characters “`--`” mark the beginning of a comment, so everything after them is ignored.) As a result, the database would return information for all user entries.

It is important to note that tautology-based attacks represent only a small subset of the different types of SQLIAs that attackers have developed. We present this type

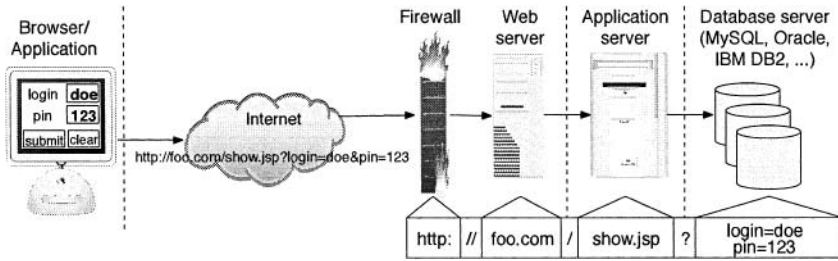


Fig. 5.1. Example of interaction between a user and a typical web application.

```
public class Show extends HttpServlet {
    ...
    1. public ResultSet getUserInfo(String login,
        String pin) {
    2.     Connection conn = DriverManager.getConnection("MyDB");
    3.     Statement stmt = conn.createStatement();
    4.     String queryString = "";

    5.     queryString = "SELECT info FROM users WHERE ";
    6.     if ((! login.equals("")) && (! pin.equals(""))) {
    7.         queryString += "login='" + login +
            "' AND pin='" + pin;
    8.     }
    9.     else {
    10.         queryString+="login='guest' ";
    11.     }
    12.     ResultSet tempSet = stmt.execute(queryString);
    13.     return tempSet;
    14. }
    ...
}
```

Fig. 5.2. Example servlet.

of attack as an example because it is fairly straightforward and intuitive. For this same reason, tautology-based attacks have been widely cited in literature and are often mistakenly viewed as the only type of SQLIAs. However, current attack techniques are not limited to only injecting tautologies. In the rest of this section, we first provide a general definition of SQLIAs and then present an overview of the currently known types of SQLIAs.

5.2.2 General Definition of SQLIA

An *SQL injection attack* occurs when an attacker changes the intended logic, semantics, or syntax of a SQL query by inserting new SQL keywords or operators. This definition includes all of the variants of SQLIAs discussed in the following subsections.

5.2.3 Variants of SQLIA

Over the past several years, attackers have developed a wide array of sophisticated attack techniques that can be used to exploit SQL injection vulnerabilities. These techniques go beyond the commonly used tautology-based SQLIA examples and take advantage of esoteric and advanced SQL constructs. Ignoring the existence of these kinds of attacks leads to the development of solutions that address the SQLIA problem only partially.

For example, SQLIA can be introduced into a program using several different types of input sources. Developers and researchers often assume that SQLIAs are only introduced via user input that is submitted as part of a web form or as a response to a prompt for input. This assumption misses the fact that any external string or input that is used to build a query string can be under the control of an attacker and represents a possible input channel for SQLIAs. It is common to see other external sources of input such as fields from an HTTP cookie or server variables being used to build a query. Since cookie values are under the control of the user's browser and server variables are often set via values from HTTP headers, these values represent external strings that can be manipulated by an attacker. In addition, *second-order injections* use advanced knowledge of a vulnerable application to introduce an attack using otherwise properly secured input sources [1]. A developer may properly escape, type-check, and filter input that comes from the user and assume it is safe. Later on, when that data is used in a different context or to build a different type of query, the previously safe input becomes an injection attack. Because there are many input sources that could lead to a SQLIA, techniques that focus on simply checking user input or explicitly enumerating all untrusted input sources are often incomplete and still leave ways for malicious input to affect the generated query strings.

Once attackers have identified an input source that can be used to exploit an SQLIA vulnerability, there are many different types of attack techniques that they can employ. Depending on the type and extent of the vulnerability, the results of these attacks can include crashing the database, gathering information about the tables in the database schema, establishing covert channels, and open-ended injection of virtually any SQL command. We briefly summarize the main techniques for performing SQLIAs using the example code from Figure 5.2. Interested readers can refer to [10] for additional information and examples of how these techniques work.

Tautologies.

The general goal of a tautology-based attack is to inject SQL tokens that cause the query's conditional statement to always evaluate to true. Although the results of this type of attack are application specific, the most common uses are to bypass authentication pages and extract data. In this type of injection, an attacker exploits a vulnerable input field that is used in the query's *WHERE* conditional. This conditional logic is evaluated as the database scans each row in the table. If the conditional represents a tautology, the database matches and returns all the rows in the table as opposed to showing only one row, as it would normally do in the absence of injection. We showed an example of this type of attack in Section 5.2.1.

Malformed Queries.

This attack technique takes advantage of overly descriptive error messages that are returned by the database when a query is rejected. Database error messages often contain useful debugging information that also allows an attacker to accurately identify which parameters are vulnerable in an application and the complete schema of the underlying database. Attackers exploit this situation by injecting SQL tokens or garbage input that causes the query to contain syntax errors, type mismatches, or logical errors. Consider our example, an attacker could try to cause a type mismatch error by injecting the following text into the *pin* input field: “ convert(int,(select top 1 name from sysobjects where xtype='u'))”.

The resulting query generated by the web application would be:

```
SELECT info FROM users WHERE login='' AND pin=
convert (int,(select top 1 name from sysobjects where xtype='u'))
```

In the attack string, the injected select query extracts the name of the first user table (*xtype='u'*) from the database's metadata table, *sysobjects*, which contains information on the structure of the database. It then converts this table name to an integer. Because the name of the table is a string, the conversion is illegal, and the database returns an error. For example, an SQL Server may return the following error: *"Microsoft OLE DB Provider for SQL Server (0x80040E07) Error converting nvarchar value 'CreditCards' to a column of data type int."*

There are two useful pieces of information in this message that aid an attacker. First, the attacker can see that the database is an SQL Server database, as the error message explicitly states this. Second, the error message reveals the string that caused the type conversion to occur (in this case, the name of the first user-defined table in the database, “CreditCards”). A similar strategy can be used to systematically extract the name and type of each column in the given table. Using this information about the schema of the database, an attacker can create more precise attacks that specifically target certain types of information. Attacks based on malformed queries are typically used as a preliminary information-gathering step for other attacks.

Union Query.

The Union Query technique refers to injection attacks in which an attacker causes the application to return data from a table that is different from the one that was intended. To this end, attackers inject a statement of the form “UNION <injected query>”. By suitably defining <injected query>, attackers can retrieve information from a specified table. The outcome of this attack is that the database returns a dataset that is the union of the results of the original query with the results of the injected query. In our example, an attacker could perform a Union Query injection by injecting the text “' UNION SELECT cardNo from CreditCards

where acctNo=10032--” into the login field. The application would then produce the following query:

```
SELECT info FROM users WHERE login='' UNION
SELECT cardNo from CreditCards where acctNo=10032 -- AND pin=
```

Assuming that there is no login equal to “” (the empty string), the original query returns the null set, and the injected query returns data from the “CreditCards” table. In this case, the database returns field “cardNo” for account “10032.” The database takes the results of these two queries, unions them together, and returns them to the application. In many applications, the effect of this attack would be that the value for “cardNo” is displayed with the account information.

Piggy-backed Queries.

In the Piggy-backed Query technique, an attacker tries to append additional queries to the original query string. If the attack is successful, the database receives and executes a query string that contains multiple distinct queries. The first query is generally the original, legal query, whereas subsequent queries are the injected, malicious queries. This type of attack can be especially harmful; attackers can use it to inject virtually any type of SQL command. In our example application, an attacker could inject the text “0; drop table users” into the *pin* input field. The application would then generate the query:

```
SELECT info FROM users WHERE login='doe' AND pin=0; drop table users
```

The database treats this query string as two queries separated by the query delimiter, “;”, and executes both. The second, malicious query causes the database to drop the *users* table in the database, which would have the catastrophic consequence of deleting all of the database users. Other types of queries can be executed using this technique, such as insertion of new users into the database or execution of stored procedures. It is worth noting that many databases do not require a special character to separate distinct queries, so simply scanning for a special character is not an effective way to prevent this attack technique.

Stored Procedures.

In this technique, attackers focus on the stored procedures that are present on the database system. Stored procedures are code that is stored in the database and run directly by the database engine. Stored procedures enable a programmer to code database or business logic directly into the database and provide an extra layer of abstraction. It is a common misconception that the use of stored procedures protects an application from SQLIAs. Stored procedures are just code and can be just as vulnerable as the application’s code. Depending on the specific stored procedures that are available on a database, an attacker has different ways of exploiting a system.

The following example demonstrates how a parameterized stored procedure can be exploited via an SQLIA. In this scenario, we assume that the query string constructed at lines 5, 7, and 9 of our example has been replaced by a call to the stored procedure defined in Figure 5.3. The stored procedure returns a boolean value to indicate whether the user’s credentials were authenticated by the database. To perform an SQLIA that exploits this stored procedure, the attacker can simply inject the text “

```

CREATE PROCEDURE DBO.isAuthenticated
    @userName varchar2, @pin int
AS
    EXEC("SELECT info FROM users
        WHERE login='" + @userName + "' and pin=" + @pin);
GO

```

Fig. 5.3. Stored procedure for checking credentials.

' ; SHUTDOWN; --" into the `userName` field. This injection causes the stored procedure to generate the following query:

```
SELECT info FROM users WHERE login=' ' ; SHUTDOWN; -- AND pin=
```

This attack works like a piggy-back attack. When the second query is executed, the database is shut down.

Inference.

Inference-based attacks create queries that cause an application or database to behave differently based on the results of the query. In this way, even if an application does not directly provide the results of the query to the attacker, it is possible to observe side effects caused by the query and deduce the results. These attacks allow an attacker to extract data from a database and detect vulnerable parameters. Researchers have reported that, using these techniques, they have been able to achieve a data extraction rate of one byte per second [2]. There are two well-known attack techniques that are based on inference: blind-injection and timing attacks.

Blind Injection: In this variation, an attacker performs queries that have a boolean result. The queries cause the application to behave correctly if they evaluate to true, whereas they cause an error if the result is false. Because error messages are easily distinguishable from normal results, this approach provides a way for an attacker to get an indirect response from the database. One possible use of blind-injection is to determine which parameters of an application are vulnerable to SQLIA. Consider again the example code in Figure 5.2. Two possible injections into the `login` field are "legalUser' and 1=0 --" and "legalUser' and 1=1 --". These injections result in the following two queries:

```
SELECT info FROM users WHERE login='legalUser' and 1=0 -- ' AND pin=
```

```
SELECT info FROM users WHERE login='legalUser' and 1=1 -- ' AND pin=
```

Now, let us consider two scenarios. In the first scenario, we have a secure application, and the input for `login` is validated correctly. In this case, both injections would return login error messages from the application, and the attacker would know that the `login` parameter is not vulnerable to this kind of attack. In the second scenario, we have a non-secure application in which the `login` parameter is vulnerable to injection. In this case, the first injection would evaluate to false, and the application would return a login-error message. Without additional information, attackers would not know whether the error occurred because the application validated the input correctly and blocked the attack attempt or because the attack itself caused the login error. However, when the attackers observe that the second query does not re-

sult in an error message, they know that the attack was successful and that the *login* parameter is vulnerable to injection.

Timing Attacks: A timing attack lets an attacker gather information from a database by observing timing delays in the database's responses. This attack is similar to blind injection, but uses a different type of observable side effect. To perform a timing attack, attackers structure their injected query in the form of an if-then statement whose branch condition corresponds to a question about the contents of the database. The attacker then uses the `WAITFOR` keyword along one of the branches, which causes the database to delay its response by a specified time. By measuring the increase or decrease in the database response time, attackers can infer which branch was taken and the answer to the injected question.

Using our example, we illustrate how to use a timing-based inference attack to extract a table name from the database. In this attack, the following text is injected into the *login* parameter:

```
legalUser' and ASCII(SUBSTRING((select top 1 name from sysobjects),1,1)) >
X WAITFOR 5 --
```

This injection produces the following query:

```
SELECT info FROM users WHERE login='legalUser' and ASCII(SUBSTRING((select top
1 name from sysobjects),1,1)) > X WAITFOR 5 -- ' AND pin=
```

In this attack, the `SUBSTRING` function is used to extract the first character of the first table's name. The attacker can then ask a series of questions about this character. In this example, the attacker is asking if the `ASCII` value of the character is greater-than or less-than-or-equal-to the value of `X`. If the value is greater, the attacker will be able to observe an additional five-second delay in the database response. The attacker can continue in this way and use a binary-search strategy to identify the value of the first character, then the second character, and so on.

Alternate Encodings. Using alternate encoding techniques, attackers modify their injection strings in a way that avoids typical signature- and filter-based checks that developers put in their applications. Alternate encodings, such as hexadecimal, `ASCII`, and `Unicode` can be used in conjunction with other techniques to allow an attack to escape straightforward detection approaches that simply scan for certain known “bad characters.” Even if developers account for alternative encodings, this technique can still be successful because alternate encodings can target different layers in the application. For example, a developer may scan for a `Unicode` or hexadecimal encoding of a single quote and not realize that the attacker can leverage a database function (e.g., `char(44)`) to encode the same character. An effective code-based defense against alternate encodings requires developers to be aware of all of the possible encodings that could affect a given query string as it passes through the different application layers. Because developing such a complete protection is very difficult in practice, attackers have been very successful in using alternate encodings to conceal attack strings. The following example attack (from [11]) shows the level of obfuscation that can be achieved using alter-

nate encodings. In the attack, the *pin* field is injected with the following string: “0; exec(0x73687574646f776e),” and the resulting query is:

```
SELECT info FROM users WHERE login='' AND pin=0; exec(char(0x73687574646f776e))
```

This example makes use of the `char()` function and ASCII hexadecimal encoding. The `char()` function takes as a parameter an integer or hexadecimal encoding of one or more characters and replaces the function call with the actual character(s). The stream of numbers in the second part of the injection is the ASCII hexadecimal encoding of the attack string. This encoded string is inserted into a query using some other type of attack profile and, when it is executed by the database, translates into the shutdown command.

5.3 Detection and Prevention of SQL Injection Attacks

AMNESIA, (Analysis for Monitoring and NEutralizing SQL Injection Attacks) is a fully-automated and general technique for detecting and preventing all types of SQLIAs. The approach works by combining static analysis and runtime monitoring. Our two key insights behind the approach are that (1) the information needed to predict the possible structure of all legitimate queries generated by a web application is contained within the application’s code, and (2) an SQLIA, by injecting additional SQL statements into a query, would violate that structure. In its static part, our technique uses program analysis to automatically build a model of the legitimate queries that could be generated by the application. In its dynamic part, our technique monitors the dynamically generated queries at runtime and checks them for compliance with the statically-generated model. Queries that violate the model represent potential SQLIAs and are reported and prevented from executing on the database.

The technique consists of four main steps. We first summarize the steps and then describe them in more detail in subsequent sections.

5.3.1 The AMNESIA Approach

Identify hotspots: Scan the application code to identify *hotspots*—points in the application code that issue SQL queries to the underlying database.

Build SQL-query models: For each hotspot, build a model that represents all the possible SQL queries that may be generated at that hotspot. A *SQL-query model* is a non-deterministic finite-state automaton in which the transition labels consist of SQL tokens (SQL keywords and operators), delimiters, and placeholders for string values.

Instrument Application: At each hotspot in the application, add calls to the runtime monitor.

Runtime monitoring: At runtime, check the dynamically-generated queries against the SQL-query model and reject and report queries that violate the model.

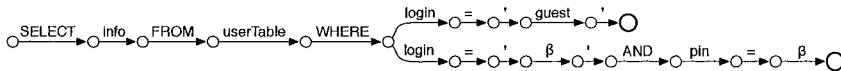


Fig. 5.4. SQL-query model for the servlet in Figure 5.2.

Identify Hotspots

In this step, AMNESIA performs a simple scan of the application code to identify hotspots. In the Java language, all interactions with the database are performed through a predefined API, so identifying all the hotspots is a trivial step. In the case of the example servlet in Figure 5.2, the set of hotspots contains a single element: the call to `stmt.execute` on line 10.

Build SQL-Query Models

In this step, we build the SQL-query model for each hotspot. We perform this step in two parts. *In the first part*, we use the Java String Analysis (JSA) developed by Christensen, Møller, and Schwartzbach [5] to compute all of the possible values for each hotspot’s query string. JSA computes a flow graph that abstracts away the control flow of the program and only represents string-manipulation operations performed on string variables. For each string of interest, the library analyzes the flow graph and simulates the string-manipulation operations that are performed on the string. The result is a Non-Deterministic Finite Automaton (NDFA) that expresses, at the character level, all possible values that the considered string variable can assume. Because JSA is conservative, the NDFA for a given string variable is an overestimate of all of its possible values.

In the second part, we transform the NDFA computed by JSA into an SQL-query model. More precisely, we perform an analysis of the NDFA that produces another NDFA in which all of the transitions are labeled with SQL keywords, operators, or literal values. We create this model by performing a depth first traversal of the character-level NDFA and grouping characters that correspond to SQL keywords, operators, or literal values. For example, a sequence of transitions labeled ‘S’, ‘E’, ‘L’, ‘E’, ‘C’, and ‘T’ would be recognized as the SQL keyword `SELECT` and grouped into a single transition labeled “SELECT”. This step is configurable to recognize different dialects of SQL. In the SQL-query model, we represent *variable strings* (i.e., strings that correspond to a variable related to some user input) using the symbol β . For instance, in our example, the value of the variable `login` is represented as β . This process is analogous to the one used by Gould, Su, and Devanbu [8], except that we perform it on NDFAs instead of DFAs.

Figure 5.4 shows the SQL-query model for the single hotspot in our example. The model reflects the two different query strings that can be generated by the code depending on the branch followed after the `if` statement at line 6 in Figure 5.2.

Instrument Application

In this step, we instrument the application by adding calls to the monitor that checks the queries at runtime. For each hotspot, the technique inserts a call to the monitor before the call to the database. The monitor is invoked with two parameters: the query string that is about to be submitted to the database and a unique identifier for the hotspot. Using the unique identifier, the runtime monitor is able to correlate the hotspot with the specific SQL-query model that was statically generated for that point and check the query against the correct model.

Figure 5.5 shows how the example application would be instrumented by our technique. The hotspot, originally at line 10 in Figure 5.2, is now guarded by a call to the monitor at line 10a.

```

...
10a. if (monitor.accepts (<hotspot ID>, queryString))
    {
10b.     ResultSet tempSet = stmt.execute(queryString);
11.     return tempSet;
    }
...

```

Fig. 5.5. Example hotspot after instrumentation.

Runtime Monitoring

At runtime, the application executes normally until it reaches a hotspot. At this point, the query string is sent to the runtime monitor, which parses it into a sequence of tokens according to the specific SQL syntax considered. In our parsing of the query string, the parser identifies empty string and empty numeric literals by their syntactic position, and we denote them in the parsed query string using ϵ . Figure 5.6 shows how the last two queries discussed in Section 5.2.1 would be parsed during runtime monitoring.

It is important to point out that our technique parses the query string in the same way that the database would and according to the specific SQL grammar considered. In other words, our technique does not perform a simple keyword matching over the query string, which would cause false positives and problems with user input that happened to match SQL keywords. For example, a user-submitted string that contains SQL keywords but is syntactically a text field, would be correctly recognized as a text field. However, if the user were to inject special characters, as in our example, to force part of the text to be evaluated as a keyword, the parser would correctly interpret this input as a keyword. Using the same parser as the database is essential because it guarantees that we are interpreting the query in the same way that the database will.

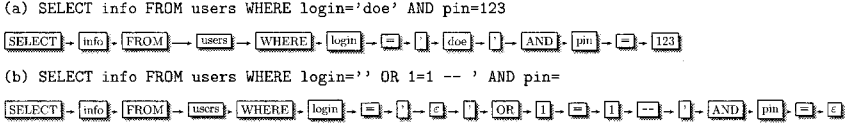


Fig. 5.6. Example of parsed runtime queries.

After the query has been parsed, the runtime monitor checks it by assessing whether the query violates the SQL-query model associated with the current hotspot. An SQL-query model is an NDFA whose alphabet consists of SQL keywords, operators, literal values, and delimiters, plus the special symbol β . Therefore, to check whether a query is compliant with the model, the runtime monitor can simply check whether the model accepts the sequence of tokens derived from the query string. A string or numeric literal (including the empty string, ε) in the parsed query string can match either β or an identical literal value in the SQL-query model.

If the model accepts the query, the monitor lets the execution of the query continue. Otherwise, the monitor identifies the query as an SQLIA. In this case, the monitor prevents the query from executing on the database and reports the attack.

To illustrate, consider again the queries shown in Figure 5.6 and recall that the first query is legitimate, whereas the second one corresponds to an SQLIA. When checking query (a), the analysis would start matching from token `SELECT` and from the initial state of the SQL-query model in Figure 5.4. Because the token matches the label of the only transition from the initial state, the automaton reaches the second state. Again, token `info` matches the only transition from the current state, so the automaton reaches the third state. The automaton continues to reach new states until it reaches the state whose two outgoing transitions are labeled “=”. At this point, the automaton would proceed along both transitions. On the upper branch, the query is not accepted because the automaton does not reach an accept state. Conversely, on the lower branch, all the tokens in the query are matched with labels on transitions, and the automaton reaches the accept state after consuming the last token in the query (“’”). The monitor can therefore conclude that this query is legitimate.

The checking of query (b) proceeds in an analogous way until token `OR` in the query is reached. Because the token does not match the label of the only outgoing transition from the current state (AND), the query is not accepted by the automaton, and the monitor identifies the query as a SQLIA.

Efficiency and limitations

For the technique to be practical, the runtime overhead of the monitoring must not affect the usability of the web application. We analyze the cost of AMNESIA’s runtime monitoring in terms of both space and time. The *space complexity* of the monitoring is dominated by the size of the generated SQL-query models. In the worst case, the size of the query models is quadratic in the size of the application. This case corresponds to the unlikely situation of a program that branches and modifies the query

string at each program statement. In typical programs, the generated automata are linear in the program size. In fact, our experience is that most automata are actually quite small with respect to the size of the corresponding application (see Table 5.1). The *time complexity* of the approach depends on the cost of the runtime matching of the query tokens against the models. Because we are checking a set of tokens against an NDFA, the worst case complexity of the matching is exponential in the number of tokens in the query (in the worst case, for each token all states are visited). In practice, however, the SQL-query models typically reduce to trees, and the cost of the matching is almost linear in the size of the query. Our experience shows that the cost of the runtime phase of the approach is negligible (see Section 5.4).

As far as limitations are concerned, our technique can generate false positives and false negatives. Although the string analysis that we use is conservative, false positives can be created in situations where the string analysis is not precise enough. For example, if the analysis cannot determine that a hard-coded string in the application is a keyword, it could assume that it is an input-related value and erroneously represent it as a β in the SQL-query model. At runtime, the original keyword would not match the placeholder for the variable, and AMNESIA would flag the corresponding query as an SQLIA. False negatives can occur when the constructed SQL query model contains spurious queries, and the attacker is able to generate an injection attack that matches one of the spurious queries. For example, if a developer adds conditions to a query from within a loop, an attacker who inserts an additional condition of the same type would generate a query that does not violate the SQL-query model. We expect these cases to be rare in practice because of the peculiar structure of SQLIAs. The attacker would have to produce an attack that directly matches either an imprecision of the analysis or a specific pattern. Moreover, in both cases, the type of attacks that could be exploited would be limited by the constraints imposed by the rest of the model that was used to match the query. It is worth noting that, in our empirical evaluation, neither false positives nor false negatives were generated (see Section 5.4).

5.3.2 Implementation

AMNESIA is the prototype tool that implements our technique for Java-based web applications. The technique is fully automated, requiring only the web application as input, and requires no extra runtime environment support beyond deploying the application with the AMNESIA library. We developed the tool in Java and its implementation consists of three modules:

Analysis module. This module implements Steps 1 and 2 of our technique. It inputs a Java web application and outputs a list of hotspots and a SQL-query model for each hotspot. For the implementation of this module, we leveraged the implementation of the Java String Analysis library by Christensen, Møller, and Schwartzbach [5]. The analysis module is able to analyze Java Servlets and JSP pages.

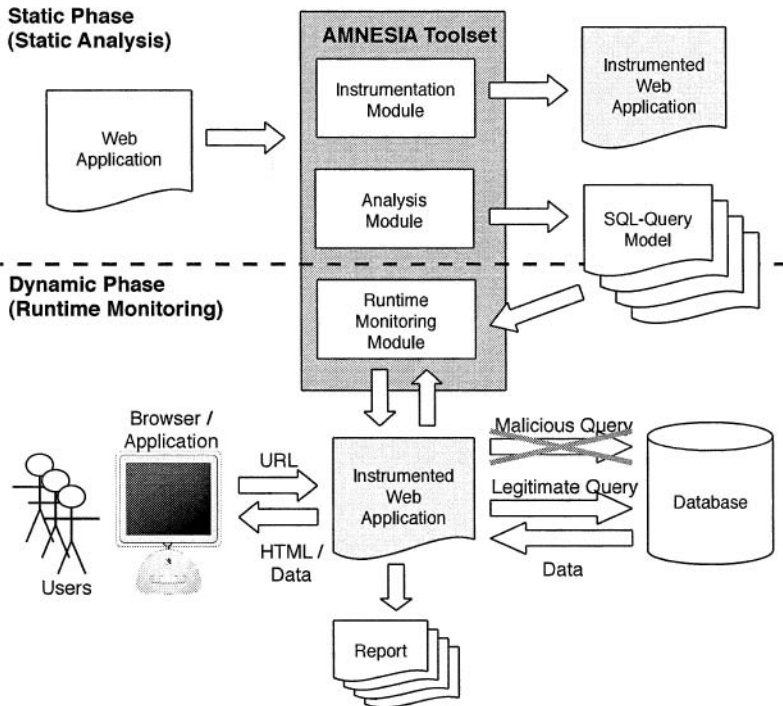


Fig. 5.7. High-level overview of AMNESIA.

Instrumentation module. This module implements Step 3 of our technique. It inputs a Java web application and a list of hotspots and instruments each hotspot with a call to the runtime monitor. We implemented this module using INSECTJ, a generic instrumentation and monitoring framework for Java developed at Georgia Tech [23].

Runtime-monitoring module. This module implements Step 4 of our technique. The module takes as input a query string and the ID of the hotspot that generated the query, retrieves the SQL-query model for that hotspot, and checks the query against the model.

Figure 5.7 shows a high-level overview of AMNESIA. In the static phase, the Instrumentation Module and the Analysis Module take as input a web application and produce (1) an instrumented version of the application, and (2) an SQL-query model for each hotspot in the application. In the dynamic phase, the Runtime-Monitoring Module checks the dynamic queries while users interact with the web application. If a query is identified as an attack, it is blocked and reported.

Once an SQLIA has been detected, AMNESIA stops the query before it is executed on the database and reports relevant information about the attack in a way that can be leveraged by developers. In our implementation of the technique for Java, we

throw an exception when the attack is detected and encode information about the attack in the exception. If developers want to access the information at runtime, they can simply leverage the exception-handling mechanism of the language and integrate their handling code into the application.

Having this attack information available at runtime is useful because it allows developers to react to an attack right after it is detected and develop an appropriate customized response. For example, developers may decide to avoid any risk and shut-down the part of the application involved in the attack. Alternatively, a developer could handle the attack by converting the information into a format that is usable by another tool, such as an Intrusion Detection System, and reporting it to that tool. Because this mechanism integrates with the application's language, it allows developers flexibility in choosing a response to SQLIAs.

Currently, the information reported by our technique includes the time of the attack, the location of the hotspot that was exploited, the attempted-attack query, and the part of the query that was not matched against the model. We are currently considering additional information that could be useful for the developer (e.g., information correlating program execution paths with specific parts of the query model) and investigating ways in which we can modify the static analysis to collect this information.

5.3.3 Implementation Assumptions

Our implementation makes one main assumption regarding the applications that it analyzes. The tool assumes that queries are created by manipulating strings in the application, that is, the developer creates queries by combining hard-coded strings and variables using operations such as concatenation, appending, and insertion. Although this assumption precludes the use of AMNESIA on some applications (e.g., applications that externalize all query-related strings in files), it is not overly restrictive and, most importantly, can be eliminated with suitable engineering.

5.4 Empirical Evaluation

The goal of our empirical evaluation is to assess the effectiveness and efficiency of the technique presented in this chapter when applied to various web applications. We used our prototype tool, AMNESIA, to perform an empirical study on a set of subjects. The study investigates three research questions:

RQ1: What percentage of attacks can AMNESIA detect and prevent that would otherwise go undetected and reach the database?

RQ2: How much overhead does AMNESIA impose on web applications at runtime?

RQ3: What percentage of legitimate accesses does AMNESIA identify as attacks?

The following sections illustrate the setup for the evaluation, and discuss the two studies that we performed to address the research questions.

Table 5.1. Subject programs for the empirical study.

<i>Subject</i> (<i>Description</i>)	<i>LOC</i>	<i>Servlets</i>	<i>Injectable</i> <i>Params</i>	<i>State</i> <i>Params</i>	<i>Hotspots</i>	<i>Automata Size</i> (<i>#nodes</i>)
Checkers (Online checkers game)	5,421	18 (61)	44	0	5	289 (2–772)
Office Talk (Purchase-order management)	4,543	7 (64)	13	1	40	40 (8–167)
Employee Directory (Online employee directory)	5,658	7 (10)	25	9	23	107 (2–952)
Bookstore (Online bookstore)	16,959	8 (28)	36	6	71	159 (2–5,269)
Events (Event tracking system)	7,242	7 (13)	36	10	31	77 (2–550)
Classifieds (Management system for classifieds)	10,949	6 (14)	18	8	34	91 (2–799)
Portal (Portal for a club)	16,453	3 (28)	39	7	67	117 (2–1,187)

5.4.1 Experiment Setup

To investigate our research questions, we leveraged a previously developed testbed for SQLIAs, which was presented in [9]. This testbed provides a set of web applications and a large set of both legitimate and malicious inputs for the applications. In the next two sections we briefly review the testbed, describe the applications it contains, and explain how the inputs were generated. Readers can refer to [9] for additional details.

Subjects

The testbed contains seven subjects. All of the subjects are typical web applications that accept user input via web forms and use that input to build queries to an underlying database. Five of the applications are commercial applications that we obtained from GotoCode (<http://www.gotocode.com>): Employee Directory, Bookstore, Events, Classifieds, and Portal. The last two applications, Checkers and OfficeTalk, were student-developed applications created for a class project. We consider them because they have been used in previous related studies [8].

In Table 5.1 we provide information about the subject applications. For each subject, the table shows: its name (*Subject*); a concise description (*Description*); its size in terms of lines of code (*LOC*); the number of accessible servlets (*Servlets*), with the total number of servlets in the application in parenthesis; the number of injectable parameters (*Injectable Params*); the number of state parameters (*State Params*); the number of hotspots (*Hotspots*); and the average size of the SQL automata generated by AMNESIA (*Automata Size*), with the minimum–maximum range in parentheses.

The table distinguishes between injectable parameters and state parameters for each application. This distinction is necessary because each type of parameter plays a different role in the application. An *injectable parameter* is an input parameter whose value is used to build part of a query that is then sent to the database. A *state parameter* is a parameter that may affect the control flow within the web application but never becomes part of a query. Because, by definition, state parameters cannot result in SQL injection, we only focus on injectable parameters for our attacks. We also distinguish between total and accessible servlets in the applications. An *accessible servlet* is a servlet that, to be accessed, only requires the user to be logged-in or does not require sessions at all. Some servlets, conversely, must have specific session data (i.e., cookies) to function properly, which considerably complicates the automation of the evaluation. Because we were able to generate enough attacks considering accessible servlets only, we did not consider the remaining servlets.

Input Generation

The sets of inputs provided by the testbed framework represent normal and malicious usages of the applications. In this section we briefly review how these sets were generated and the types of inputs they contain.

In a preliminary step, we identified all of the servlets in each web application and the corresponding parameters that could be submitted to the servlet. Each parameter was identified as either an injectable or state parameter. State parameters must be handled specially because they often determine the behavior of the application. Without a correct and meaningful value assigned to them, the application fails and no attack can be successful. Lastly, we identified the expected type of each injectable parameter. This information helps us in identifying potential attacks that can be used on the parameter and in generating legitimate inputs.

The set of *attack strings* was generated independently using commercial penetration testing techniques. For this task, we leveraged the services of a Masters-level student at Georgia Tech who worked for a local software-security company. The student is an experienced programmer who has developed commercial-level penetration tools for detecting SQL-injection vulnerabilities. In addition, the student was not familiar with our technique, which reduced the risk of developing a set of attacks biased by the knowledge of the approach and its capabilities.

To define the initial set of attack strings, the student used a combination of sources, including (1) exploits developed by commercial penetrating teams to take advantage of SQL-injection vulnerabilities, (2) online sources of vulnerability reports, such as US-CERT (<http://www.us-cert.gov/>) and CERT/CC Advisories (<http://www.cert.org/advisories/>), and (3) information extracted from several security-related mailing lists. The resulting set of attack strings contained thirty unique types of attacks. All types of attacks reported in literature (e.g., [1]) were represented in this set with the exception of attacks that take advantage of overly-descriptive database error messages and second-order injections. We excluded these kinds of attacks because they are multi-phase attacks that require intensive human intervention to interpret the attacks' partial results.

The student generated two sets of inputs for each application. The first set contained normal or legitimate inputs for the application. We call this set *LEGIT*. The second set contained malicious inputs, that is, strings that would result in an SQLIA. We call this set *ATTACK*. To populate the *LEGIT* set, the student generated, for each servlet, different combinations of legitimate values for each injectable parameter. State parameters were assigned a meaningful and correct value. To populate the *ATTACK* set, a similar process was used. For each accessible servlet in the application the student generated the Cartesian product of its injectable parameters using values from the initial attack strings and legitimate values. This approach generated a large set of potentially malicious inputs, which we used as the *ATTACK* set.

5.4.2 Study 1: Effectiveness

In the first study, we investigated **RQ1**, the effectiveness of our technique in detecting and preventing SQLIAs. We analyzed and instrumented each application using AMNESIA and ran all of the inputs in each of the applications' *ATTACK* sets. For each application, we measured the percentage of attacks detected and reported by AMNESIA. (As previously discussed, when AMNESIA detects an attack, it throws an exception, which is in turn returned by the web application. Therefore, it is easy to accurately detect when an attack has been caught.)

The results for this study are shown in Table 5.2. The table shows, for each subject, the number of unsuccessful attacks (*Unsuccessful*),² the number of successful attacks (*Successful*), and the number of attacks detected and reported by AMNESIA (*Detected*) in absolute terms and as a percentage over the total number of successful attacks, in parentheses. As the table shows, AMNESIA achieved a perfect score. For all subjects, it was able to correctly identify all attacks as SQLIAs, that is, it generated no false negatives.

Table 5.2. Results of Study 1.

<i>Subject</i>	<i>Unsuccessful</i>	<i>Successful</i>	<i>Detected</i>
Checkers	1195	248	248 (100%)
Office Talk	598	160	160 (100%)
Employee Directory	413	280	280 (100%)
Bookstore	1028	182	182 (100%)
Events	875	260	260 (100%)
Classifieds	823	200	200 (100%)
Portal	880	140	140 (100%)

² Because the applications performed input validation, they were able to block a portion of the attacks without the attack reaching AMNESIA's monitor.

5.4.3 Study 2: Efficiency and Precision

In the second study, we investigated **RQ2** and **RQ3**. To investigate **RQ2**, the efficiency of our technique, we ran all of the inputs in the LEGIT sets on the uninstrumented web applications and measured the response time of the applications for each web request. We then ran the same inputs on the versions of the web applications instrumented by AMNESIA and again measured the response time. The difference in the two response times corresponds to the overhead imposed by our technique.

We found that the overhead imposed by our technique is negligible and, in fact, barely measurable, averaging about 1 millisecond. Note that this time should be considered an upper bound on the overhead, as our implementation was not optimized. These results confirm our expectations. Intuitively, the time for the network access and the database transaction completely dominates the time required for the runtime checking. As the results show, our technique is efficient and can be used without significantly affecting the response time of a web application.

To investigate **RQ3**, the rate of false positives generated by our technique, we simply assessed whether AMNESIA identified any legitimate query as an attack. The results of the assessment were that AMNESIA correctly identified all such queries as legitimate queries and reported no false positives.

5.4.4 Discussion

The results of our study are very encouraging. For all subjects, our technique was able to correctly identify all attacks as SQLIAs, while allowing all legitimate queries to be performed. In other words, for the cases considered, our technique generated no false positives and no false negatives. The lack of false positives and false negatives is promising and provides evidence of the viability of the technique.

In our study, we did not compare our results with alternative approaches against SQLIAs because most of the existing automated approaches address only a subset of the possible SQLIAs. (For example, the approach in [8] is focused on type safety, and the one in [25] focuses only on tautologies.) Therefore, we can conclude analytically that such approaches would not be able to identify many of the attacks in our test bed.

As for all empirical studies, there are some threats to the validity of our evaluation, mostly with respect to external validity. The results of our study may be related to the specific subjects considered and may not generalize to other web applications. To minimize this risk, we used a set of real web applications (except for the two applications developed by students teams) and an extensive set of realistic attacks. Although more experimentation is needed before drawing definitive conclusions on the effectiveness of the technique, the results we obtained so far are promising.

5.5 Related Approaches

There has been a wide range of techniques proposed to counter SQLIAs. However, when compared to AMNESIA, these solutions have several limitations and shortcomings. In this section we review and discuss the main approaches against SQLIAs.

Defensive Programming.

Developers have proposed a range of code-based development practices to counter SQLIAs. These techniques generally focus on proper input filtering, such as escaping potentially harmful characters and rigorous type-checking of inputs. Many of these approaches are summarized in Reference [11]. In general, a rigorous and systematic application of these techniques is an effective solution to the problem. However, in practice, the application of such techniques is human-based and is therefore less than ideal. For example, many SQLIA vulnerabilities that have been discovered in various applications correspond to cases where the applications contained input-validation operations, but the validation was inadequate. The situation is further complicated because attackers continue to find new attack strings or subtle variation on old attacks that are able to avoid the checks programmers put in place. Lastly, retroactively fixing vulnerable legacy applications using defensive coding practices is complicated, labor-intensive, and error-prone.

Two widely suggested “SQLIA remedies” merit specific mention. Both of them initially appear to offer viable solutions to the SQLIA problem, but do not correctly address it. The first remedy consists of simply checking user input for malicious keywords. This approach would clearly result in a high rate of false positives because an input field could legally contain words that match SQL keywords (i.e. “FROM”, “OR”, or “AND”). The second remedy is to use stored procedures for database access. The ability of stored procedures to prevent SQLIAs is dependent on their implementation. The mere fact of using stored procedures does not protect against SQLIA. Interested readers may refer to Section 5.2 and to References [1, 15, 18, 19] for examples of how SQLIAs can be performed in the presence of stored procedures.

Two approaches, SQL DOM [17] and Safe Query Objects [6], use encapsulation of database queries to provide a safe and reliable way to access databases. These techniques offer an effective way to avoid the SQLIA problem by changing the query-building process from one that uses string concatenation to a systematic one that uses a type-checked API. (In this sense, SQL DOM and Safe Query Objects can be considered instances of defensive coding.) Although these techniques are as effective as AMNESIA, they have the drawback that they require developers to learn and use a new programming paradigm or query-development process.

In general, defensive coding has not been successful in completely preventing SQLIA. While improved coding practices can help mitigate the problem, they are limited by the developer’s ability to generate appropriate input validation code and recognize all situations in which it is needed. AMNESIA, being fully automated, can provide stronger guarantees about the completeness and accuracy of the protections put in place.

General Techniques Against SQLIAs.

Security Gateway [22] uses a proxy filter to enforce input validation rules on the data that reaches a web application. Using a descriptor language, developers create filters that specify constraints and transformations to be applied to application parameters as they flow from the web page to the application server. By creating appropriate filters, developers can block or transform potentially malicious user input. The effectiveness of this approach is limited by the developer's ability to (1) identify all the input streams that can affect the query string and (2) determine what type of filtering rules should be placed on the proxy.

WAVES [12] is a penetration testing tool that attempts to discover SQLIA vulnerabilities in web applications. This technique improves over normal penetration-testing techniques by using machine learning to guide its testing. However, like all penetration testing techniques, it can not provide guarantees of completeness.

Valeur and colleagues [24] propose the use of an Intrusion Detection System (IDS) to detect SQLIAs. Their IDS is based on a machine learning technique that is trained using a set of typical application queries. The technique builds models of normal queries and then monitors the application at runtime to identify queries that do not match the model. The fundamental limitation of learning based techniques is that they can not provide guarantees about their detection abilities because their success is dependent on the use of an optimal training set. Without such a set, this technique could generate a large number of false positives and negatives.

Boyd and Keromytis propose SQLrand, an approach that uses key-based randomization of SQL instructions [4]. In this approach, SQL code injected by an attacker would result in a syntactically incorrect query because it was not specified using the randomized instruction set. While this technique can be very effective, there are several practical drawbacks to this approach. First, the security of the key may be compromised by looking at the error logs or messages. Furthermore, the approach imposes a significant infrastructure overhead because it requires the integration of an encryption proxy for the database.

Static Detection Techniques.

JDBC-Checker is a technique for statically checking the type correctness of dynamically generated SQL queries [8]. Although this technique was not originally intended to address SQLIA, it can detect one of the root causes of SQL-injection vulnerabilities—improper type checking of input. In this sense, JDBC-Checker is able to detect and help developers eliminate some of the code that allows attackers to exploit type mismatches. However, JDBC-Checker cannot prevent other types of SQLIAs that produce syntactically and type correct queries.

Wassermann and Su propose an approach that uses static analysis combined with automated reasoning to verify that the SQL queries generated in the application layer cannot contain a tautology [25]. The scope of this technique is limited, in that it can only address one type of SQLIAs, namely tautology-based attacks, whereas AMNESIA is designed to address all types of SQLIAs.

Taint-based Approaches.

Two similar approaches have been proposed by Nguyen-Tuong *et al.* [20] and Pietraszek and Berghel [21]. These approaches modify a PHP interpreter to track precise taint information about user input and use a context sensitive analysis to detect and reject queries if untrusted input has been used to create certain types of SQL tokens. In general, these taint-based techniques have shown much promise in their ability to detect and prevent SQLIAs. The main drawback of these approaches concerns their practicality. First, identifying all sources of tainted user input in highly-modular web applications introduces problem of completeness. Second, accurately propagating taint information may result in high runtime overhead for the web applications. Finally, the approach relies on the use of a customized version of the runtime system, which affects portability.

Huang and colleagues define WebSSARI, a white-box approach for detecting input-validation-related errors, that is based on information-flow analysis [13]. This approach uses static analysis to check information flows against preconditions for sensitive functions. The analysis detects where preconditions are not satisfied and suggests filters and sanitization functions that can be automatically added to the application to satisfy the preconditions. The primary drawbacks of this technique are the assumptions that (1) preconditions for sensitive functions can be adequately and accurately expressed using their type system and (2) forcing input to pass through certain types of filters is sufficient to consider it trusted. For many types of functions and applications, these assumptions do not hold.

Livshits and Lam [14] use a static taint analysis approach to detect code that is vulnerable to SQLIA. This approach checks whether user input can reach a hotspot and flags this code for developer intervention. A further extension to this work, Securify [16], detects vulnerable code and automatically adds calls to a sanitization function. This automated defensive coding practice, while effective in some cases, would not prevent all types of SQLIAs. In particular, it would not prevent SQLIAs that inject malicious text into numeric non-quoted fields.

5.6 Conclusion

SQLIAs have become one of the more serious and harmful attacks on database-driven web applications. They can allow an attacker to have unmitigated access to the database underlying an application and, thus, the power to access or modify its contents. In this article, we have discussed the various types of SQLIAs known to date and presented AMNESIA, a fully automated technique and tool for detecting and preventing SQLIAs. AMNESIA uses static analysis to build a model of the legitimate queries that an application can generate and runtime monitoring to check the dynamically generated queries against this model. Our empirical evaluation, performed on commercial applications using a large number of realistic attacks, shows that AMNESIA is a highly effective technique for detecting and preventing SQLIAs. Compared to other approaches, AMNESIA offers the benefit of being fully automated and is general enough to address all known types of SQLIAs.

Acknowledgments

This material is based upon work supported by NSF award CCR-0209322 to Georgia Tech and by the Department of Homeland Security and United States Air Force under Contract No. FA8750-05-2-0214. Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the United States Air Force. Jeremy Viegas developed our test bed infrastructure.

References

1. C. Anley. Advanced SQL Injection In SQL Server Applications. White paper, Next Generation Security Software Ltd., 2002.
2. C. Anley. (more) Advanced SQL Injection. White paper, Next Generation Security Software Ltd., 2002.
3. D. Aucsmith. Creating and maintaining software that resists malicious attack. http://www.gtisc.gatech.edu/aucsmith_bio.htm, September 2004. Distinguished Lecture Series.
4. S. W. Boyd and A. D. Keromytis. SQLrand: Preventing SQL injection attacks. In *Proceedings of the 2nd Applied Cryptography and Network Security (ACNS) Conference*, pages 292–302, June 2004.
5. A. S. Christensen, A. Møller, and M. I. Schwartzbach. Precise analysis of string expressions. In *Proc. 10th International Static Analysis Symposium, SAS '03*, volume 2694 of *LNCS*, pages 1–18. Springer-Verlag, June 2003. Available from <http://www.brics.dk/JSA/>.
6. W. R. Cook and S. Rai. Safe Query Objects: Statically Typed Objects as Remotely Executable Queries. In *Proceedings of the 27th International Conference on Software Engineering (ICSE 2005)*, 2005.
7. T. O. Foundation. Top ten most critical web application vulnerabilities, 2005. <http://www.owasp.org/documentation/topten.html>.
8. C. Gould, Z. Su, and P. Devanbu. Static Checking of Dynamically Generated Queries in Database Applications. In *Proceedings of the 26th International Conference on Software Engineering (ICSE 04)*, pages 645–654, 2004.
9. W. G. Halfond and A. Orso. AMNESIA: Analysis and Monitoring for NEutralizing SQL-Injection Attacks. In *Proceedings of the IEEE and ACM International Conference on Automated Software Engineering (ASE 2005)*, Long Beach, CA, USA, Nov 2005.
10. W. G. Halfond, J. Viegas, and A. Orso. A Classification of SQL-Injection Attacks and Counter Techniques. Technical report, Georgia Institute of Technology, August 2005.
11. M. Howard and D. LeBlanc. *Writing Secure Code*. Microsoft Press, Redmond, Washington, second edition, 2003.
12. Y. Huang, S. Huang, T. Lin, and C. Tsai. Web Application Security Assessment by Fault Injection and Behavior Monitoring. In *Proceedings of the 11th International World Wide Web Conference (WWW 03)*, May 2003.
13. Y. Huang, F. Yu, C. Hang, C. H. Tsai, D. T. Lee, and S. Y. Kuo. Securing Web Application Code by Static Analysis and Runtime Protection. In *Proceedings of the 12th International World Wide Web Conference (WWW 04)*, May 2004.

14. V. B. Livshits and M. S. Lam. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Usenix Security Symposium*, August 2005.
15. O. Maor and A. Shulman. SQL Injection Signatures Evasion. White paper, Imperva, April 2004. http://www.imperva.com/application_defense_center/white_papers/sql_injection_signatures_evasion.html.
16. M. Martin, V. B. Livshits, and M. S. Lam. Finding Application Errors and Security Flaws Using PQL: a Program Query Language. In *Proceedings of the ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, October 2005.
17. R. McClure and I. Krüger. SQL DOM: Compile Time Checking of Dynamic SQL Statements. In *Proceedings of the 27th International Conference on Software Engineering (ICSE 05)*, pages 88–96, 2005.
18. S. McDonald. SQL Injection: Modes of attack, defense, and why it matters. White paper, GovernmentSecurity.org, April 2002. <http://www.governmentsecurity.org/articles/SQLInjectionModesofAttackDefenceandWhyItMatters.php>.
19. S. McDonald. SQL Injection Walkthrough. White paper, SecuriTeam, May 2002. <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>.
20. A. Nguyen-Tuong, S. Guarnieri, D. Greene, J. Shirley, and D. Evans. Automatically Hardening Web Applications Using Precise Tainting Information. In *Twentieth IFIP International Information Security Conference (SEC 2005)*, May 2005.
21. T. Pietraszek and C. V. Berghe. Defending Against Injection Attacks through Context-Sensitive String Evaluation. In *Proceedings of Recent Advances in Intrusion Detection (RAID2005)*, 2005.
22. D. Scott and R. Sharp. Abstracting Application-level Web Security. In *Proceedings of the 11th International Conference on the World Wide Web (WWW 2002)*, pages 396–407, 2002.
23. A. Seesing and A. Orso. InsECTJ: A Generic Instrumentation Framework for Collecting Dynamic Information within Eclipse. In *Proceedings of the eclipse Technology eXchange (eTX) Workshop at OOPSLA 2005*, pages 49–53, San Diego, USA, October 2005.
24. F. Valeur, D. Mutz, and G. Vigna. A Learning-Based Approach to the Detection of SQL Attacks. In *Proceedings of the Conference on Detection of Intrusions and Malware and Vulnerability Assessment (DIMVA)*, Vienna, Austria, July 2005.
25. G. Wassermann and Z. Su. An Analysis Framework for Security in Web Applications. In *Proceedings of the FSE Workshop on Specification and Verification of Component-Based Systems (SAVCBS 2004)*, pages 70–78, 2004.